

HelixLLM

HelixLLM

Адзін бінарны файл, шэсць рэжымаў — OpenAI- і Anthropic-сумяшчальны інферэнс ад ноўтбука да кластара з некалькіх хостаў.

Кароткае змесціва

HelixLLM — гэта распрацаваная для прадпрыемстваў размеркаваная сістэма LLM на базе Go: адзін бінарны файл з сістэмай рэжымаў, якая маштабуецца ад аднахоставай распрацоўкі да шматхоставай эксплуатацыі. Яна прапануе поўнасю OpenAI- і Anthropic-сумяшчальныя API праз HTTP/3, лакальны інферэнс llama.cpp, ланцужок аўтаматычнага пераключэння паміж некалькімі правайдэрамі з ацэнкай якасці, канвеер RAG і сістэму агентаў ReAct.

Кароткае апісанне

HelixLLM — гэта адзіны бінарны файл, размеркаваная сістэма LLM на базе Go. Яна падтрымлівае OpenAI- і Anthropic-сумяшчальныя API праз HTTP/3, запуская лакальны інферэнс llama.cpp, аўтаматычна знаходзіць і ацэньвае бясплатныя воблачныя правайдэры для стварэння ланцужка пераключэння, а таксама ўключае канвеер ведаў RAG і агента ReAct з выклікам інструментаў — і ўсё гэта можна разгарнуць у шасці рэжымах.

Падрабязнае апісанне

HelixLLM — гэта размеркаваная сістэма LLM прамысловага ўзроўню, створаная на базе Go з выкарыстаннем Gin. Яе галоўная асаблівасць у тым, што адзін артэфакт абслугоўвае любы маштаб. Яна кампілюецца ў адзіны бінарны файл, рэжым якога вызначае пры разгортванні, чым менавіта ён будзе: запусціце яго ў рэжыме full для ўсёўключальнага экзemplяра

на ноўтбуку або размеркавайце абавязкі паміж рэжымамі gateway, brain, knowledge, agents і control на некалькіх хостах — той жа код, проста перагрупуваны, а не перапісаны, ад камп'ютара распрацоўшчыка да вытворчага кластара.

Сістэма свабодна падтрымлівае два дыялекты: поўнасьцю OpenAI- і Anthropic-сумяшчальныя API, таму існуючыя кліенты SDK з абодвух экасістэм працуюць без змен, усё падаецца праз HTTP/3 (QUIC) з аўтаматычным пераключэннем на HTTP/2 і падтрымкай TLS 1.3. Лакальны інферэнс ажыццяўляецца праз llama.cpp з падтрымкай CUDA, Metal і ROCm, таму адна і тая ж зборка паскараецца на абсталяванні Nvidia, Apple і AMD. Асабліва вылучаецца ланцужок аўтаматычнага пераключэння паміж некалькімі правайдэрамі, які ператварае характэрную ненадзейнасць бясплатных воблачных інферэнсаў у кіраваны, самааднаўляльны рэсурс: HelixLLM аўтаматычна знаходзіць бясплатныя мадэлі ад 7+ воблачных правайдэраў (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), ацэньвае іх праз LLMsVerifier кожныя пяць хвілін і маршрутызуе запыты праз ранжыраваны ланцужок з аўтаматычным пераключэннем пры памылках 429/5xx — заўсёды з лакальным llama.cpp як гарантаваным апошнім сродкам, каб запыт ніколі не правальваўся праз адсутнасць даступнага правайдэра.

Акрамя чыстага інферэнсу, HelixLLM з'яўляецца поўнай платформай для прыкладанняў: канвеер ведаў RAG (індэксацыя, фрагментацыя, эмбедынг, пошук vector) і сістэма агентаў ReAct з выклікам інструментаў, сеансамі гутарак і інтэграцыяй з RAG уключаны ў той жа бінарны файл. Сістэма рэжымаў апраўдвае сябе і на ўзроўні сеткі: у рэжыме full усе ўзроўні звязваюцца праз прамыя ўнутрыпрацэсныя выклікі Go без сеткавых накладных выдаткаў, а той жа бінарны файл, размеркаваны паміж хостамі, каардынуе працу праз gRPC, SSE і Kafka. Дадаткова сістэма падтрымлівае перагаворы аб фармаце змесціва праз Brotli/gzip, стрымінг SSE, сумяшчальны з фарматамі OpenAI і Anthropic пабайтова, аўтэнтыфікацыю праз API-ключы і JWT з абмежаваннем частаты запытаў, метрыкі Prometheus, трасіроўку OpenTelemetry і шырокі набор вытворчых модуляў Go.

Змест

Чаму мы яго стварылі

Камандам патрэбны інферэнс, які з'яўляецца пераносным, сумяшчальным са стандартамі і ўстойлівым — без перапісвання кліентаў ці прывязкі да аднаго пастаўшчыка ці адной машыны. HelixLLM быў створаны, каб адзін і той жа бінарны мог працаваць лакальна для распрацоўкі і

маштабавацца да шматхоставага вытворчага кластара, выкарыстоўваючы дыялекты OpenAI і Anthropic, да якіх кліенты ўжо звыклі.

Чаму гэта пераварот у гульні

Ён аб'ядноўвае ўвесь стэк інферэнсу — шлюз, лакальны інферэнс, рэзервовы варыянт у воблаку, RAG і агенты — у адзін бінарны, кіраваны пераключальнікам рэжымаў. Такім чынам, архітэктара, якую вы разгортваеце, становіцца пытаннем часу выканання, а не праектам пераплатформіравання. І ён ператварае тое, што раней было слабым месцам, у перавагу: надзейнасць воблачнага пастаўшчыка становіцца прыярытэтай задачай, якая бесперапынна маніторыцца і кіруецца ацэньвальным ланцужком самааднаўлення. Ён пераразмяркоўвае пастаўшчыкоў кожныя некалькі хвілін і заўсёды дэградуе да гарантаванага лакальнага інферэнсу. Гэта адкрывае магчымасць стварэння адзінага канцавога пункта, на які сапраўды можна разлічваць — сумяшчальнага са стандартамі, пераноснага з ноўтбука на кластар і не здольнага згаснуць з-за таго, што адзін з апстрымавых пастаўшчыкоў абмежаваў ліміты ці зламаўся.

Што новага

- Адзіны бінарны файл з шасцю рэжымамі, які працуе як усеабдымна, так і ў якасці размеркаваных роляў — прамыя ўнутрыпрацэсныя выклікі Go у рэжыме full, gRPC/SSE/Kafka у размеркаваных рэжымах. Такім чынам, змена тапалогіі разгортвання не патрабуе змены кода ці дадатковых сеткавых выдаткаў.
- Ацэньвальны ланцужок аўтаматычнага пераключэння паміж некалькімі бясплатнымі пастаўшчыкамі (7+), якія бесперапынна ранжыруюцца па LLMsVerifier з аўтаматычным пераключэннем пры памылках 429/5xx і гарантаваным апошнім рэзервовым варыянтам — llama.cpp. Бясплатныя рэсурсы ператвараюцца ў надзейныя.
- Дваічны паверхні, сумяшчальныя з OpenAI і Anthropic, якія падаюцца праз HTTP/3 з аўтаматычным пераключэннем на HTTP/2. Такім чынам, кліенты з абодвух экасістэм падключаюцца без змен.
- Лакальны інферэнс, які падтрымлівае CUDA, Metal і ROCm з аднаго кодавага база — адзін і той жа будынак працуе з паскарэннем на абсталяванні Nvidia, Apple і AMD.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолелі

- **Маштабаванне з аднаго хоста на некалькі без перапісвання.** Большасць сістэм ствараюць жорсткую мяжу паміж «лакальнай распрацоўкай» і «размеркаванай вытворчасцю», і пераход праз яе азначае перабудову архітэктury. Мы знішчылі гэту мяжу з дапамогай сістэмы рэжымаў у адзіным бінарным файле: тыя ж пласты звязваюцца праз прамыя ўнутрыпрацэсныя выклікі ў рэжыме full і празрыста пераключаюцца на gRPC/SSE/Kafka у размеркаваных рэжымах. Такім чынам, маштабаванне становіцца пытаннем змены канфігурацыі, а не партавання.
- **Ненадзейныя, абмежаваныя бясплатныя воблачныя пастаўшчыкі.** Бясплатны інферэнс хуткі, пакуль не з'явіцца памылка 429 ці ён не знікне на сярэдзіне запыту. Мы зрабілі яго надзейным, аўтаматычна выяўляючы даступныя мадэлі, ацэньваючы іх па LLMsVerifier, праактыўна адсочваючы загатоўкі абмежаванняў, каб перанакіроўваць запыты ад пастаўшчыкоў, якія вот-вот будуць абмежаваны, і аўтаматычна пераключаючыся на llama.cpp у якасці апошняга рэзервовага варыянта. Такім чынам, нестабільнасць пула ніколі не даходзіць да кліента.
- **Сумяшчальнасць кліентаў у дзвюх экасістэмах.** Перапісванне кліентаў для адаптацыі да новага бэкэнда інферэнсу — гэта непрымальна. Мы рэалізавалі абодва фарматы OpenAI і Anthropic API — уключаючы іх адметныя фарматы стрымінгу SSE — такім чынам, што SDK з абодвух лагераў накіроўваюцца на HelixLLM і проста працуюць.

Змесціва

Тэхналагічны стэк

- **Go + Gin** — абраны таму, што адзіны бінарны файл з прыярытэтам канкурэнтнасці робіць усю сістэму рэжымаў магчымай: адзін збор, які можа быць лакальным серверам ці вузлом кластэра. Ён змяшчае ўсю сістэму і HTTP-слой шлюзу.
- **HTTP/3 (QUIC) + TLS 1.3 з рэзервовым HTTP/2** — абраны для сучаснага, нізкалатэнтнага і ўстойлівага да разрываў злучэння транспарту, які прадстаўлены як серверная паверхня з аўтаматычным перагаворным працэсам, каб кліенты, якія не падтрымліваюць QUIC, ціха пераклучаліся на HTTP/2.
- **llama.cpp (CUDA/Metal/ROCm)** — абраны для партатыўнага лакальнага выносу, які паскараецца на Nvidia, Apple і AMD-бэкэндах з аднаго кодавага база; ён

таксама служыць гарантаваным апошнім рэзервовым пастаўшчыком, які не дазваляе ланцужку адмоваў абарваць працу.

- **LLMsVerifier** — абраны, каб ператварыць пытанне “які пастаўшчык зараз лепшы” у лічбу; ён ацэньвае і ранжыруе ланцужок рэзервовых воблачных пастаўшчыкоў кожныя 5 хвілін, каб маршрутызацыя арыентавалася на рэальную якасць, а не на састарэлыя здагадкі.
- **Воблачныя пастаўшчыкі (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together)** — абраны для выкарыстання бясплатных рэсурсаў розных апстрымаў; аўтаматычна выяўляюцца і ранжыруюцца ў адзіны ланцужок адмоваў, каб ні адзін пастаўшчык не стаў кропкай адмовы.
- **gRPC + SSE + Kafka** — абраны як транспартныя сродкі для размеркаваных разгортванняў: gRPC для выклікаў паміж сэрвісамі, SSE для струменевай перадачы, а Kafka — для дэкапліраванага патоку падзей паміж ролямі.
- **Vector store / embeddings** — абраны для забеспячэння канвеера ведаў RAG ад пачатку да канца: загрузка, фрагментацыя, эмбедынг і пошук па дакументах, якія забяспечваюць грунтоўнасць адказаў мадэлі.
- **Prometheus + OpenTelemetry** — абраны для мэтрык і размеркаванага трасіравання, якія адсочваюць запыт незалежна ад таго, якія рэжымы разгорнуты.
- **Падмадулі vasic-digital Go** — абраны для паўторнага выкарыстання пераправяраных прымітываў вытворчай інфраструктуры замест іх паўторнага стварэння, што забяспечвае аднастайнасць асновы сістэмы з шырэйшым стэкам.

Стан і адкрытасць інфармацыі

- **Стан: бэта.** Функцыянальная, актыўна развіваемая сістэма размеркаванага выносу.
- **Ліцэнзія: пакуль не вызначана.** Рэпазіторый не змяшчае інфармацыі аб ліцэнзіі ў метададзеных (licenseInfo — null) — гэта НЕПРАВЕРАНА і павінна быць вырашана да таго, як будзе абвешчана ліцэнзія.
- Кананічны рэпазіторый на дадзены момант знаходзіцца па адрасе github.com/HelixDevelopment/llm; шлях HelixLLM перанакіроўвае на яго. Паказчыкі пакрыцця тэстаў і колькасці падмадуляў у README пададзены аўтарам сістэмы.

Прыярытэтны ўзровень: Helix-прыярытэтны.